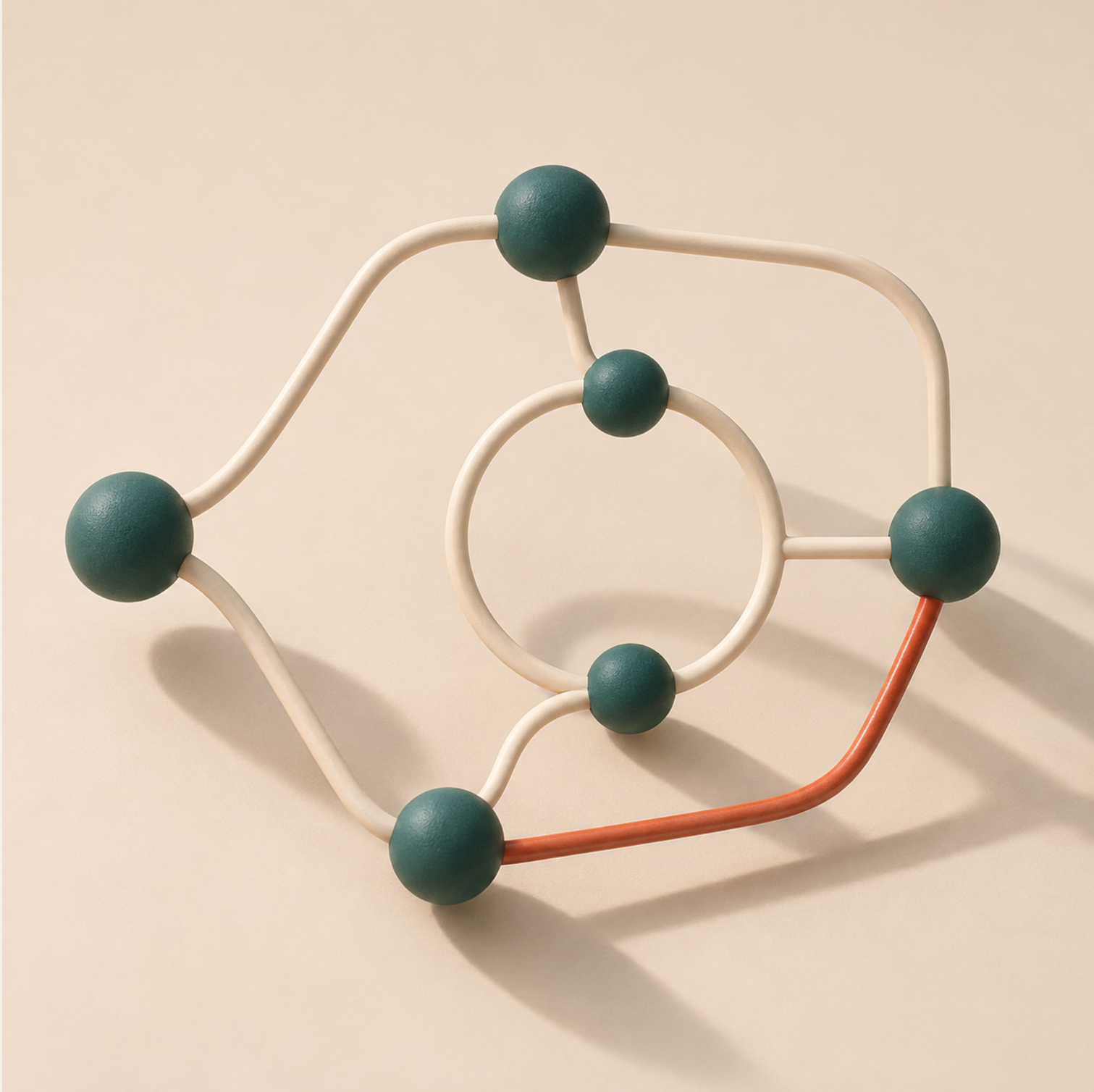
THE VISUAL GUIDE

Graph Engineering

Build agents that work together.

One idea per page.
Six patterns you can use.

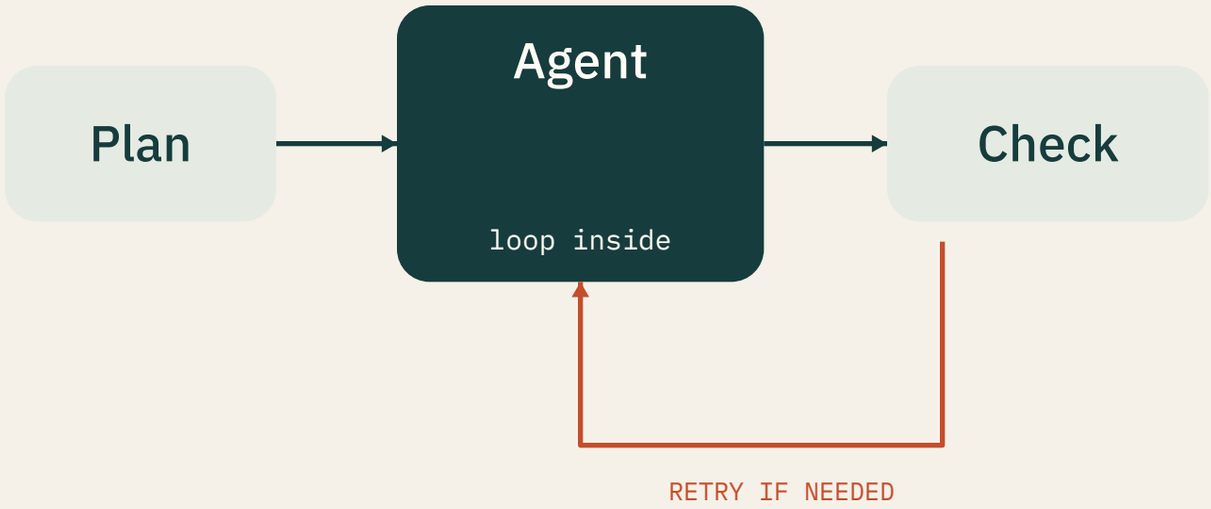
UNDERSTAND 02
BUILD 07
OPERATE 16
START 20



A loop lives inside the graph.

An agent repeats actions.
A graph coordinates the work.

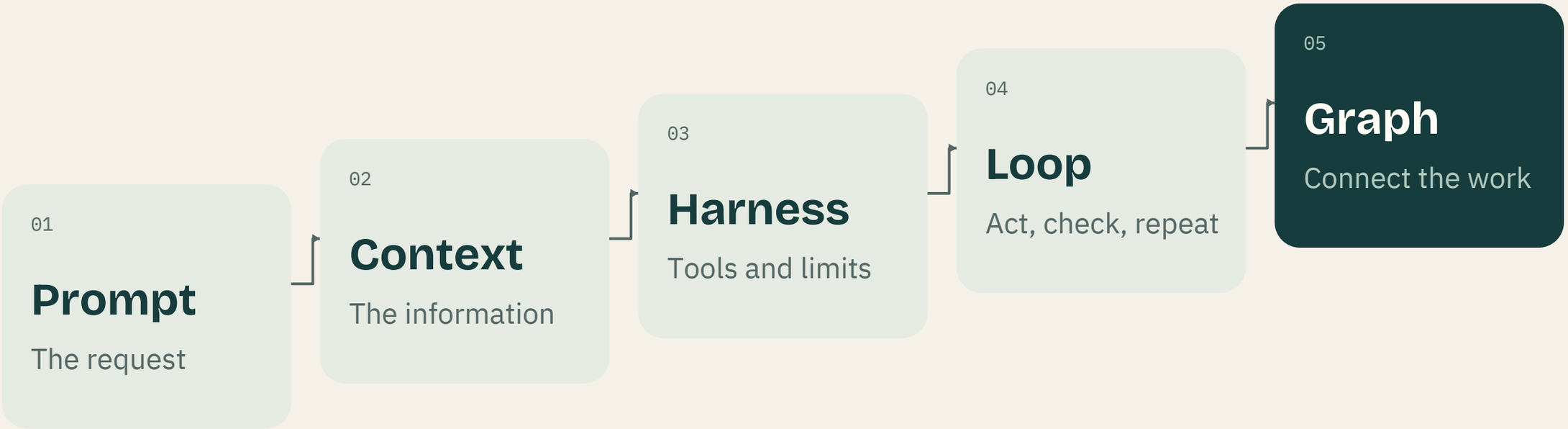
An agent is a model that uses tools to pursue a goal. Its loop is the repeated cycle of acting and checking.



- The graph decides who runs next, with what information and rules.

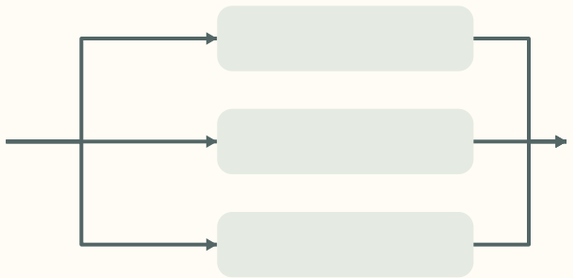
Five things you design.

Each layer gives the next one something to build on.



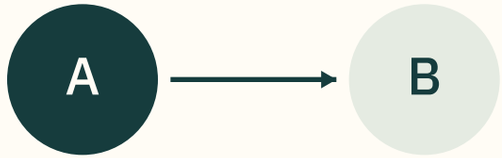
- A better graph still needs good prompts, useful context and capable nodes.

Add structure where it helps.



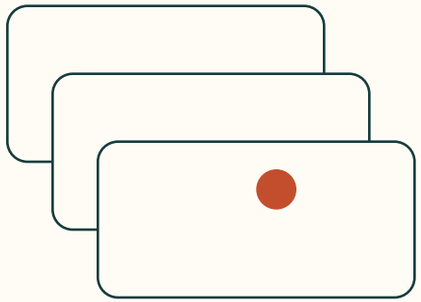
Parallel work

Separate tasks can run together.



Fresh review

A separate check can catch missed errors.

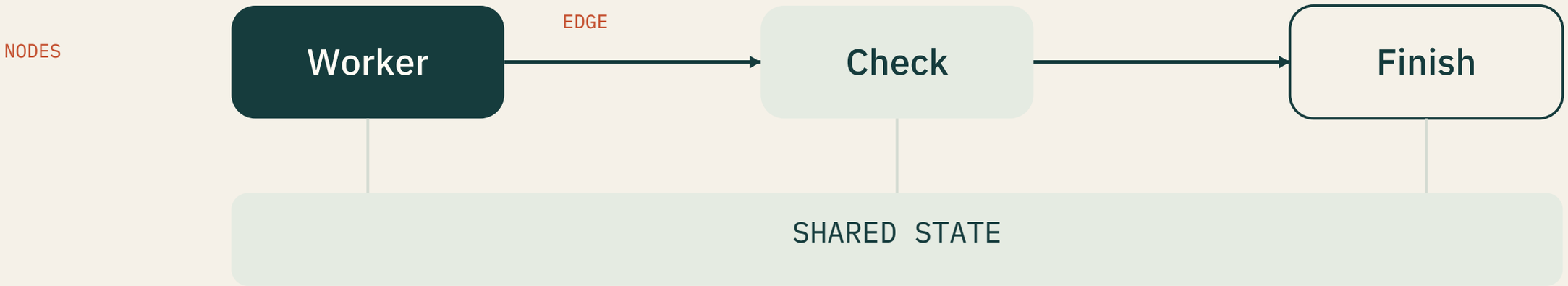


Durable state

Saved progress survives interruptions.

- Start with one useful loop. Add coordination when you can name the benefit.

Four parts. One system.



Nodes

Agent, code, tool or person.

Edges

Choose what runs next.

State

Remember the facts.

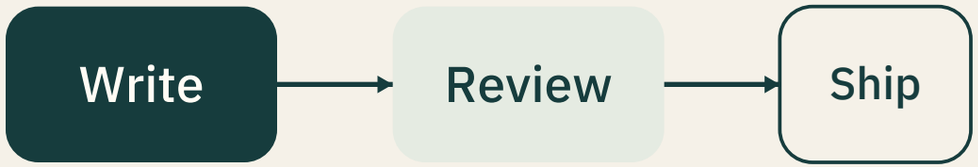
Contracts

Inputs, outputs and limits.

Work graphs and data graphs.

EXECUTION GRAPH

What happens next?



Nodes are jobs.
Edges are handoffs.

KNOWLEDGE GRAPH

How are facts related?



Nodes are things.
Edges are relationships.

Give every node a clear job.

A contract says what a node may do and what it must return.

01 / JOB

Review one patch.

02 / INPUT

Patch + requirements.

03 / OUTPUT

Findings + evidence.

04 / TOOLS

Read files; run checks.

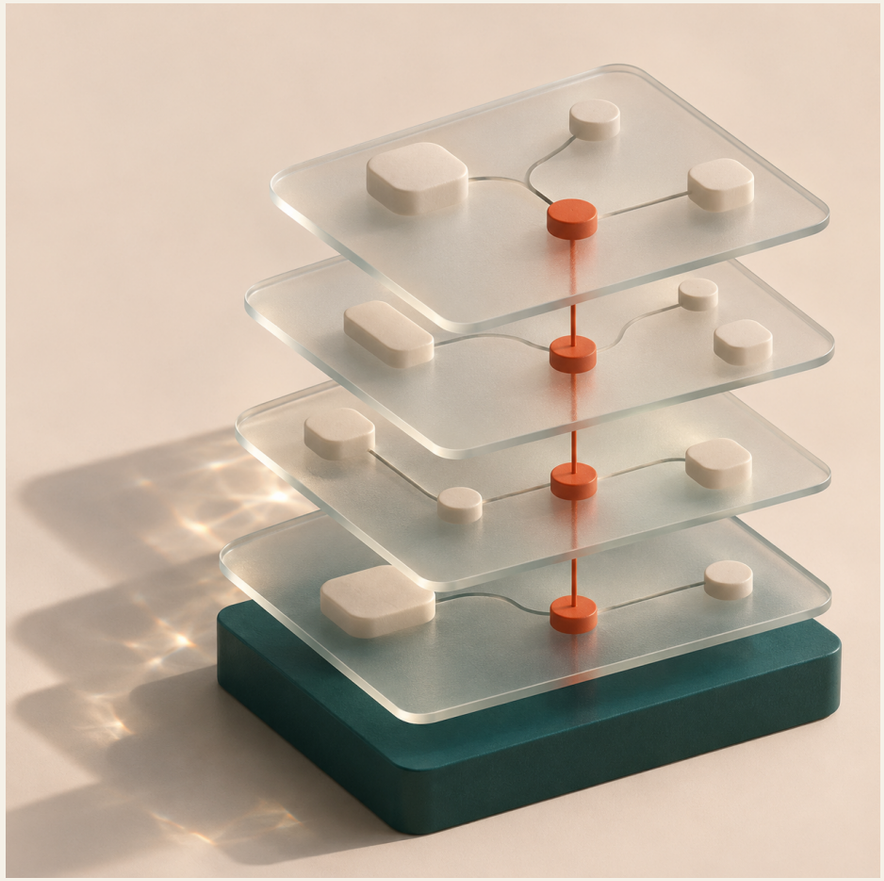
05 / LIMITS

Timeout; retry cap; cost cap.

06 / DONE

Every requirement checked.

Keep a record the system can read.



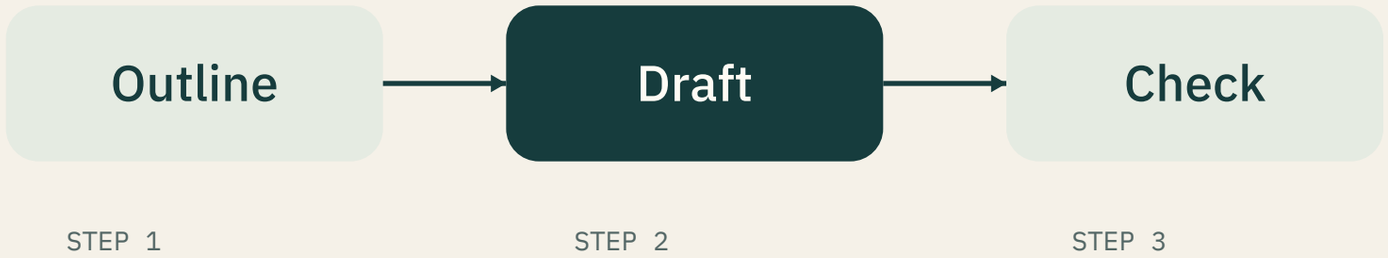
TYPED STATE = NAMED FIELDS WITH DEFINED FORMATS

`task`	What success means
`artifact`	The current draft or patch
`findings`	Problems + supporting evidence
`checks`	Pass/fail results
`budget`	Time, spend and retries left
`approval`	Pending, approved or denied

Give each field one writer or a rule for combining updates.

Chain: work in order.

One result becomes
the next step's input.



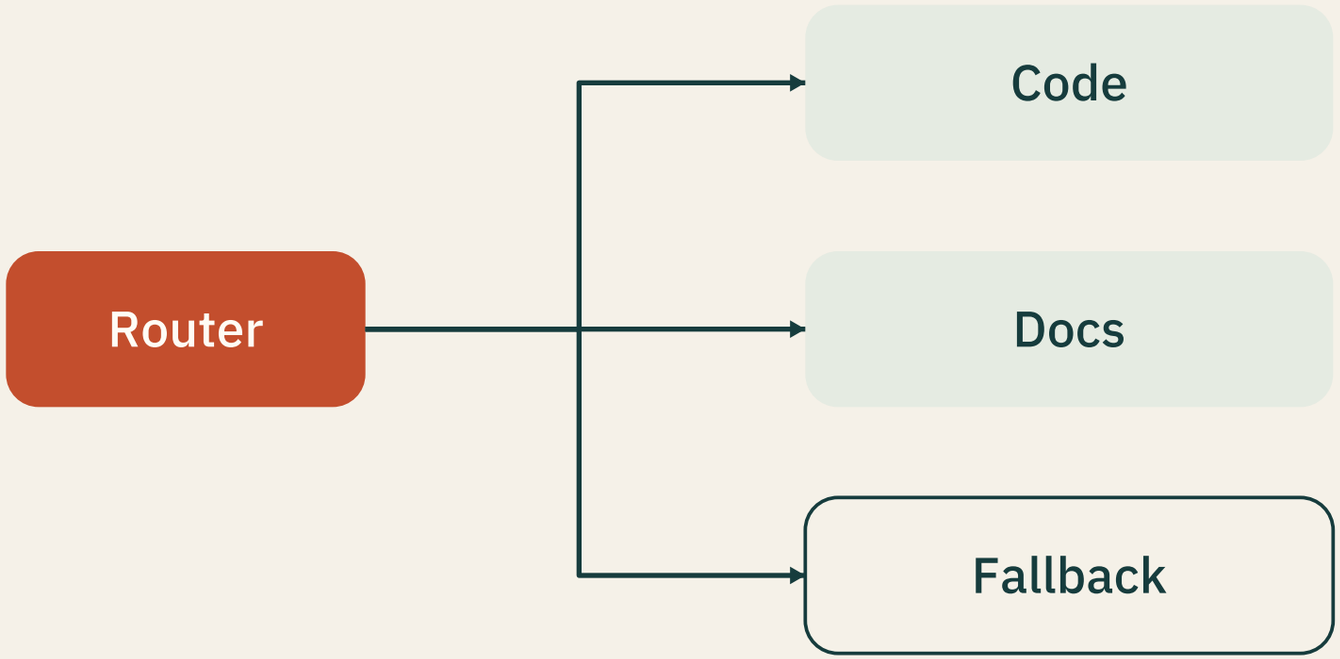
USE WHEN

The sequence is known.

- Check each handoff before the next step uses it.

Route: choose the right path.

Classify the request.
Send it to one path.

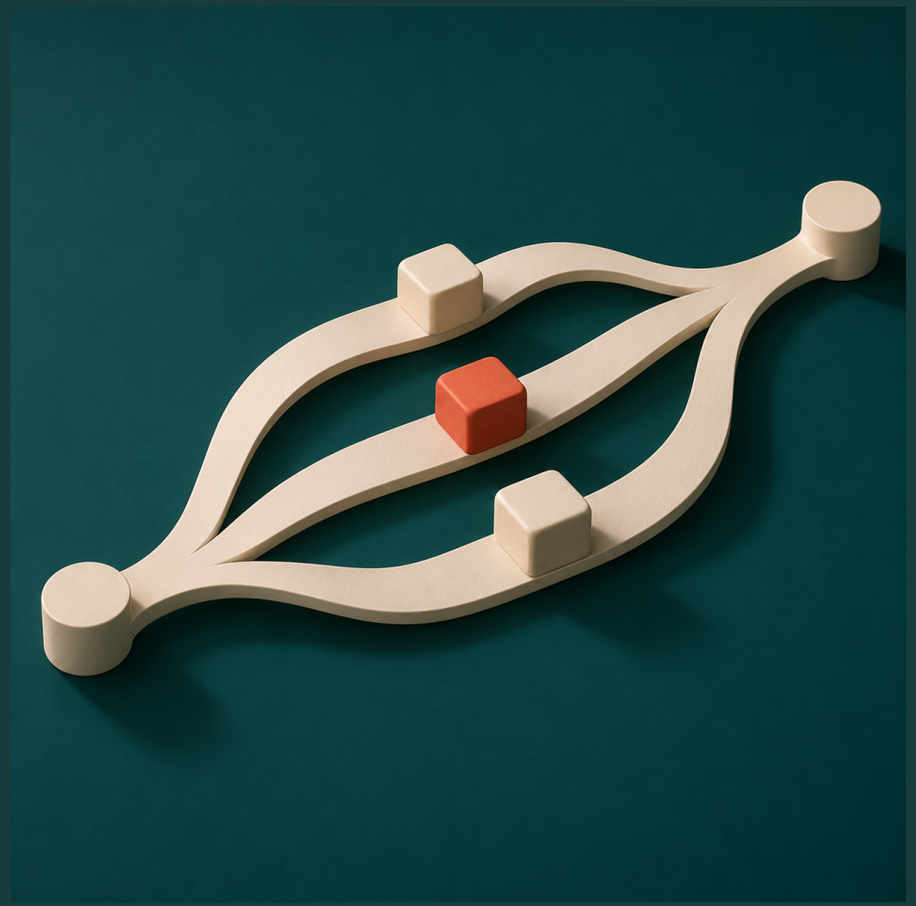


USE WHEN

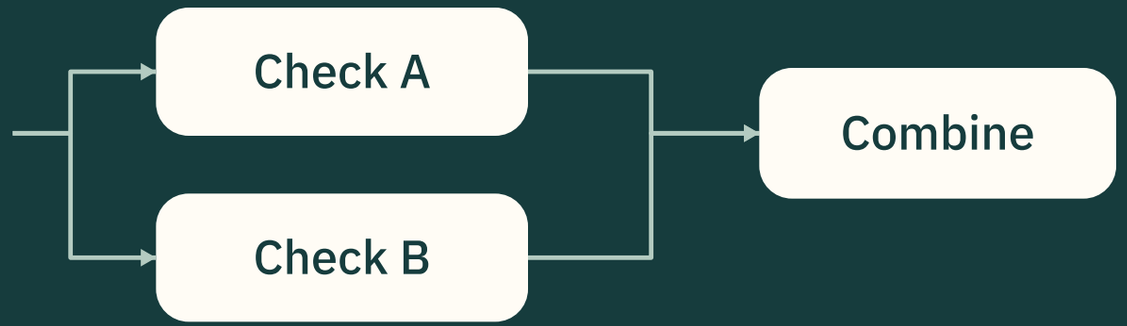
Different inputs need different handling.

- Give uncertain inputs a fallback path.

Parallelize: split, then join.



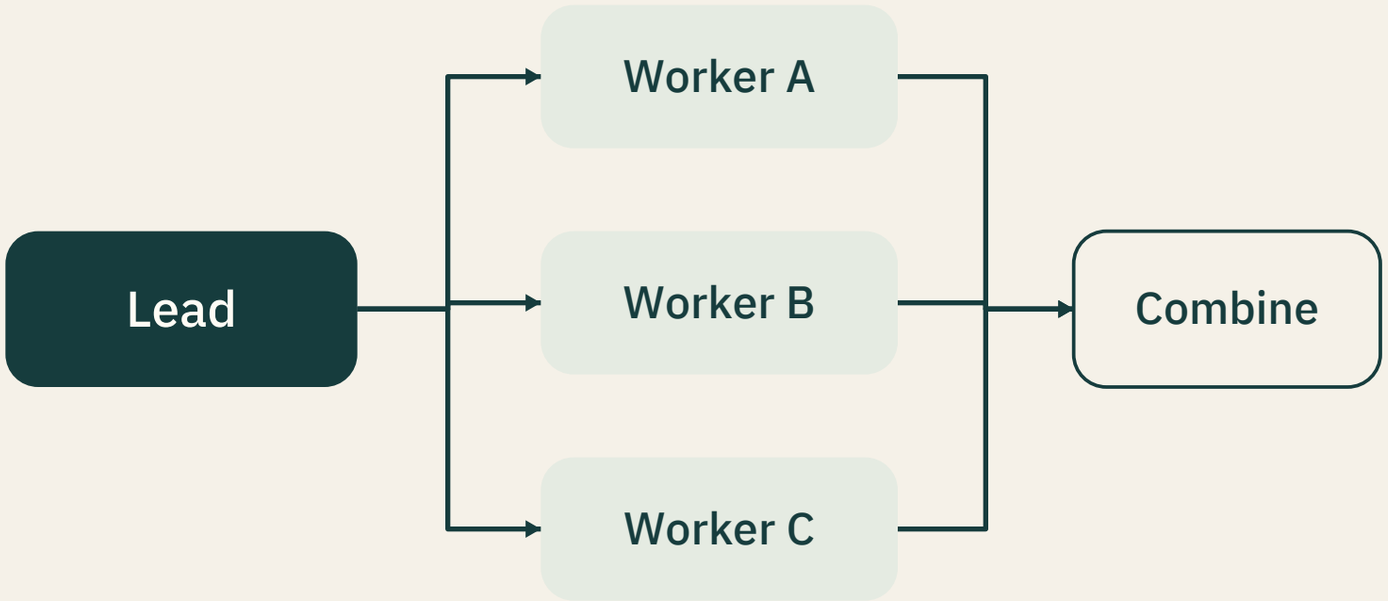
Independent tasks can run together.



Define what the join does when a branch fails.

Delegate: let the task shape the team.

A lead agent creates jobs, then combines the results.



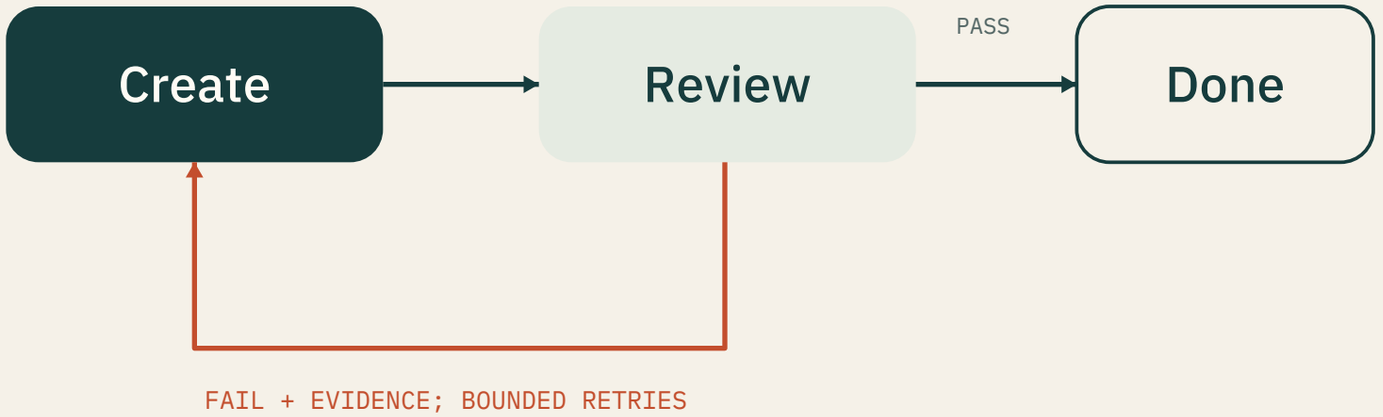
USE WHEN

Subtasks emerge as the work unfolds.

- Give workers separate jobs and a shared spending limit.

Review: improve with evidence.

A reviewer checks the result and returns specific failures.



USE WHEN

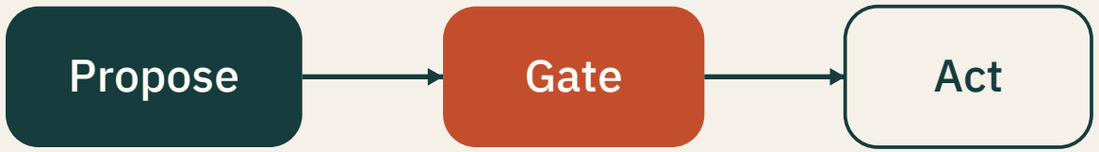
Quality has a clear test or scoring rule.

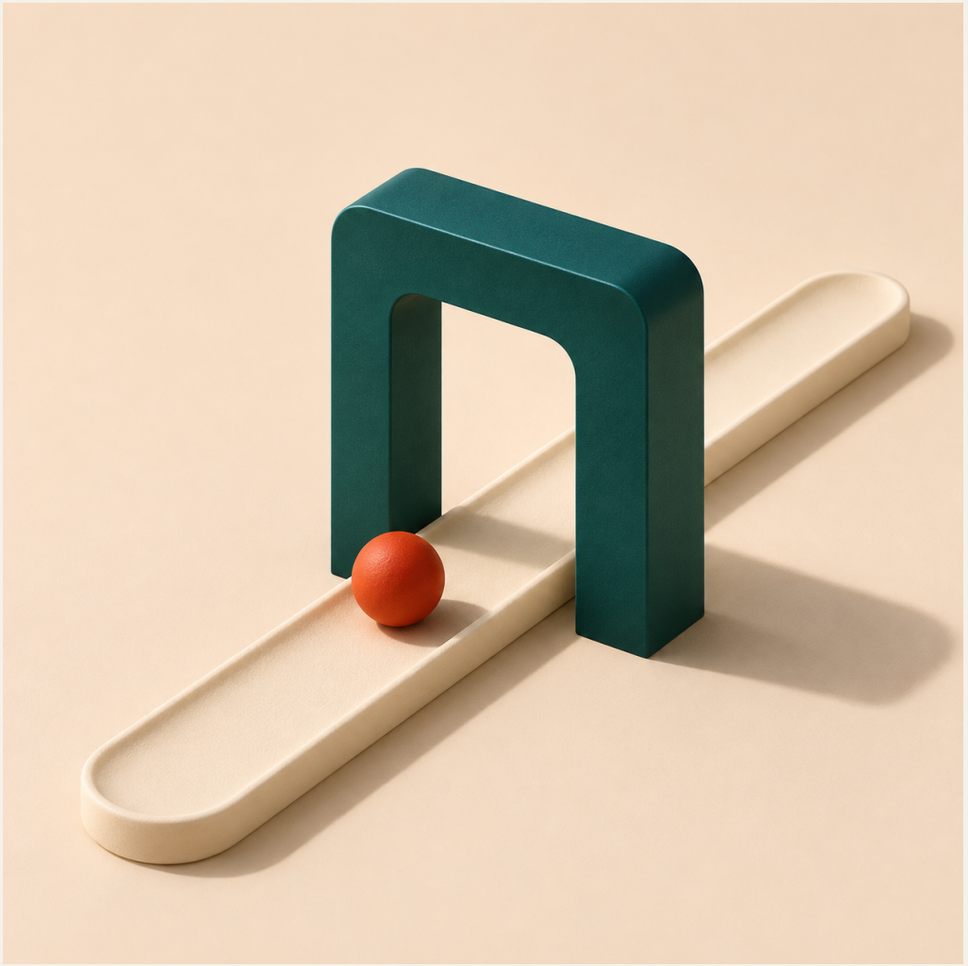
- Fresh context helps. Tests and different methods add stronger evidence.

Gate: pause before the action.

A proposal becomes an action only after the required checks.

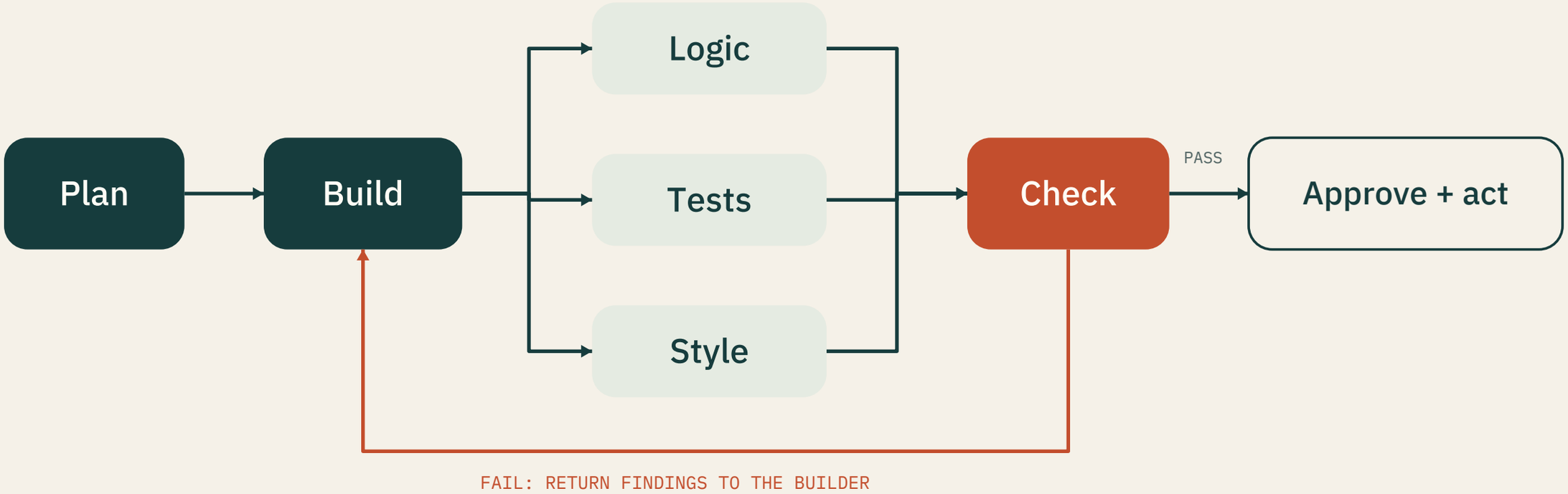
Use code for fixed rules.
Use a person when judgment or authorization is required.





One feature. A small, useful graph.

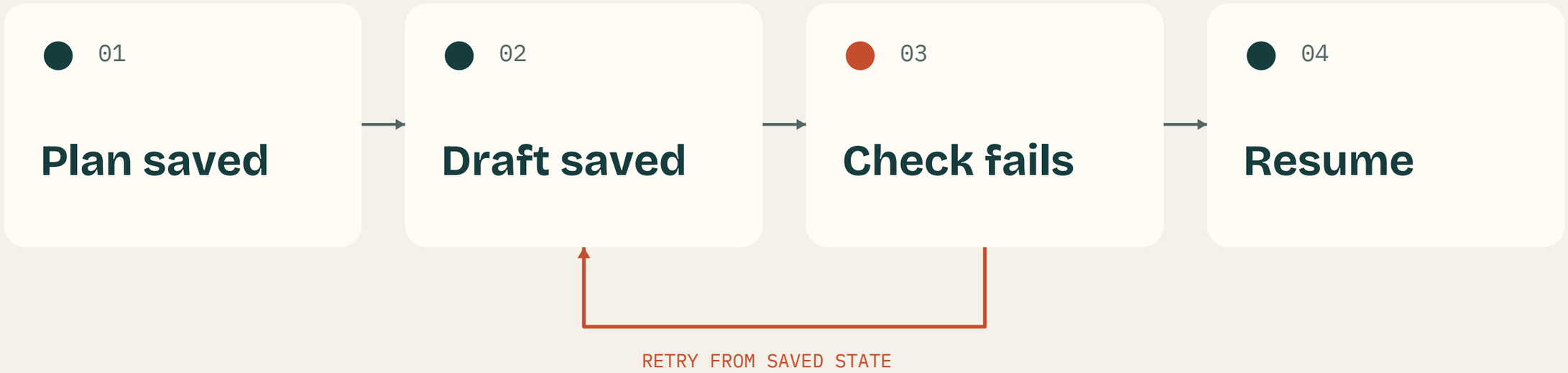
Each reviewer checks the same patch in a separate context.



Pass = required tests succeed and blocking findings are resolved.

Save progress. Resume deliberately.

A checkpoint is a saved snapshot of the run.



- Restoring state does not undo a sent message, payment or deployment.

Make the failure path explicit.

WHEN THIS HAPPENS		BUILD THIS IN
A retry loop never ends.	→	Enforce time and retry limits.
Two nodes overwrite a field.	→	Assign one writer, or combine writes.
An external action repeats.	→	Use a stable action ID and deduplicate.
A route is uncertain.	→	Send it to a defined fallback.

Less waiting can mean more spending.

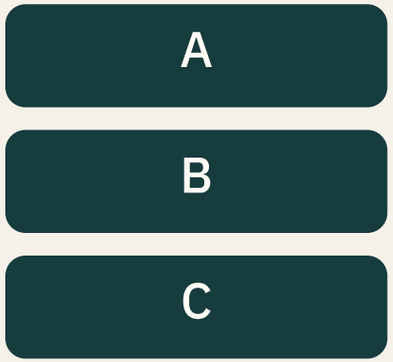
Illustration: three independent jobs, two minutes each.

SEQUENTIAL



6 min

PARALLEL



~2 min + coordination

Still three jobs. The work did not disappear.

- Set limits for total spend, workers and retries. Measure real runs.

Measure the whole result.

01

Quality

Did the output meet the requirements?

02

Time

How long until a usable result arrived?

03

Cost

What did each successful task cost?

A trace records the steps a run took.

- **Compare with a simple baseline. Repeat trials; inspect failures and traces.**

Choose the shape of the work.

KEEP ONE LOOP

One owner.

The path is uncertain.
Steps depend closely on each other.
A short run is cheap to repeat.

ADD GRAPH STRUCTURE

Clear handoffs.

Independent jobs can run together.
Required checks must happen.
Long runs need pause and resume.

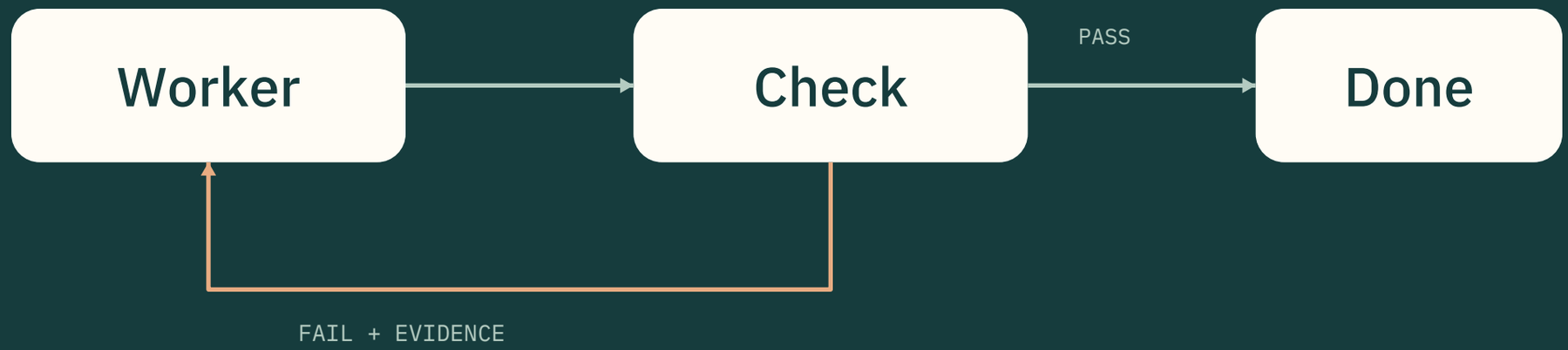
Use the simplest structure that reliably completes the task.

Try the graph before you automate it.

1. **Name the output.** Write one sentence that defines success.
2. **List the jobs.** Separate work that needs different inputs or checks.
3. **Draw the handoffs.** Pass a clear artifact from one job to the next.
4. **Run it by hand.** Use a fresh chat or separate context for each role.
5. **Compare results.** Keep the structure only if it earns its overhead.

Build the smallest useful version.

One worker. One independent check.
A bounded path back.



Plain functions can be enough. Add a workflow engine when pause/resume or coordination needs it.

Six rules worth remembering.

01 **Start simple.**

Prove one loop can do useful work.

02 **Name the handoffs.**

Define inputs, outputs and who owns them.

03 **Separate the checks.**

Judge evidence, not the agent's confidence.

04 **Save useful state.**

Plan for crashes, pauses and retries.

05 **Bound the work.**

Limit time, spend, workers and attempts.

06 **Gate the action.**

Enforce required checks before acting.

Go deeper when you need to.

PATTERNS

Building effective agents

Anthropic

GRAPH STRUCTURE

Graph API overview

LangChain documentation

SAVED STATE

Persistence

LangChain documentation

PRODUCTION LESSONS

How we built our multi-agent research system

Anthropic

Everything needed to use this guide is on the preceding pages.